GMD AI-Assisted Harmonization System

Infrastructure Requirements — Request for ITS Assessment

GMD Harmonization Team

May 2026

Table of contents

PURPOSE OF THIS DOCUMENT
This document describes the infrastructure components required to build and operate an AI-assisted harmonization pipeline for the Global Monitoring Database (GMD). It is written for a non-technical audience and is intended to support a structured conversation with ITS. Each component is described in plain language, including what it does, why it is needed, the Azure-native service we believe fits the requirement, and the specific question we need ITS to answer. A consolidated list of all ITS questions appears at the end of this document.

---

1 What We Are Building — and Why It Needs Infrastructure

The GMD harmonization process currently relies on expert analysts who manually read raw household survey documentation, write transformation scripts in Stata, and apply their judgment to convert country-specific variables into a standardized GMD format. This process is time-consuming, difficult to scale, and heavily dependent on individual expertise that is not always documented or transferable.

We are building an AI-assisted pipeline that changes this process in three concrete ways:

- **Draft:** The AI reads new survey documentation and produces a structured Harmonization Specification — a document that says exactly how each raw variable should be transformed.
- **Approve:** A human expert reviews and approves that specification. Nothing runs in production without sign-off.
- **Execute:** A deterministic (non-AI) execution engine reads the approved specification and produces the harmonized data. The same approved specification always produces the same output.

Critically, this system does not replace human judgment. It automates the first draft and the mechanical execution, while keeping all methodological decisions firmly in human hands.

The infrastructure described in this document is what makes the above possible. Without it, the AI has no place to read from, no place to write to, no way to access the language model, and no way to run.

---

2 Division of Responsibilities

Before describing the components, it is important to be explicit about what we are asking ITS to do, and what our team will do ourselves. This distinction matters because it affects what we need ITS to provision and how we need those resources to be configured.

Table 1: Division of Responsibilities

ITS Is Responsible For	GMD Team Is Responsible For
Provisioning and securing the cloud environment	Writing and deploying all pipeline code (agents, harmonization logic, prompts)
Connecting services so they can communicate securely	Designing the AI workflow and variable mapping logic
Ensuring the environment meets WBG data and security policies	Populating and maintaining the rules and historical databases
Maintaining the underlying infrastructure (backups, patches, uptime)	Reviewing, approving, and overriding AI-generated specifications
Approving which AI and cloud services are available in the WBG tenant	Monitoring outputs and iterating on the system

⚠ **KEY REQUIREMENT — TEAM ACCESS**
For this division to work, our team must be able to deploy updated code, monitor job logs, and iterate on the pipeline without filing an ITS ticket for every change. We are asking ITS to configure the compute environment so that the GMD team has direct, self-service access to deploy and monitor. This is a specific configuration requirement, not an afterthought.

---

3 Infrastructure Components

The system requires six infrastructure components. Each is described below in terms of what it does, why we need it, what Azure service we believe fits the requirement, and what we need to confirm with ITS.

3.1 Component 1 — File Storage

"The shared filing cabinet where all raw materials live"

3.1.1 What It Does

File storage is the central location where all raw files are kept — household survey data files, questionnaire PDFs, codebooks, legacy Stata harmonization scripts, and final harmonized output datasets. Think of it as a secure, cloud-based filing cabinet that all parts of the system can access.

3.1.2 Why We Need It

All pipeline components — the AI service, the execution engine, the quality assurance checks — need to read and write files. They need a shared, consistent, network-accessible location to do this. Files cannot live on a personal laptop or a shared drive that is only reachable from certain machines.

We also need two logical tiers of storage within this component:

- **Raw originals:** The original survey files exactly as received, preserved for audit and reproducibility purposes.
- **Processed extracts:** Structured information derived from those files (e.g., parsed DDI metadata, extracted variable lists) that the pipeline actually works with day-to-day.

3.1.3 Azure Service

Azure Blob Storage — a standard, scalable, policy-compliant object storage service available in Azure. This is not exotic technology and ITS almost certainly has approved configurations for it.

3.1.4 Special Consideration: DDI Files and the Microdata Library API

The World Bank Microdata Library (https://microdata.worldbank.org) provides machine-readable DDI (Data Documentation Initiative) metadata files for many of our surveys via a public API. DDI files are structured XML documents that already contain variable labels, question text, response categories, and other metadata in a standardized format — making them far more reliable as AI inputs than unstructured PDFs.

Where a DDI file is available, it will be our preferred ingestion path. This means the system needs to make outbound API calls to the Microdata Library. We need ITS to confirm whether such outbound calls are permitted from within the Azure environment, or whether there is an internal network path to the Microdata Library.

> **ITS QUESTION 1**
> Is there an approved Azure Blob Storage environment where we can store restricted household survey microdata? If so, what access controls and data classification requirements apply? Separately: can our compute environment make outbound API calls to `microdata.worldbank.org`?

3.2 Component 2 — Rules Database

"The official GMD rulebook — always the final authority"

3.2.1 What It Does

This is a structured, queryable database that stores the canonical definitions of every GMD variable: its name, label, valid value range, data type, whether it is mandatory or optional, what other variables are derived from it, and what validation rules apply. It is the official rulebook for the entire system.

3.2.2 Why We Need It

Every time the AI drafts a harmonization specification, it must check the official definition of the target variable. Every time the execution engine runs, it validates output against these definitions. This database is what makes those checks possible.

It must be a proper database — not a spreadsheet or flat file — because multiple system components need to query it simultaneously, and because it must enforce internal consistency: two conflicting definitions of the same variable must be impossible.

3.2.3 Design Principle: Schema Always Wins

This database is always authoritative. When the AI consults historical precedents (see Component 4) and those precedents suggest something that contradicts this database, the database wins. This is a foundational design rule.

3.2.4 Azure Service

Azure SQL Database or PostgreSQL on Azure, depending on what ITS has already approved and provisioned. Both are standard relational database services. Either will meet our needs; we defer to ITS on which is preferred in our environment.

> **ITS QUESTION 2**
> What is the standard approved relational database service in the WBG Azure environment — Azure SQL, PostgreSQL, or another option? Is there an existing instance we could use, or would we need a new one provisioned?

3.3 Component 3 — AI Language Model Service

"The brain that reads surveys and drafts harmonization specifications"

> **! MOST CRITICAL QUESTION FOR ITS**
> This component requires a yes-or-no answer from ITS before we can finalize any other part of the AI design. Please see the ITS question at the end of this section.

3.3.1 What It Does

When a new survey arrives, an AI language model reads the survey documentation (DDI file, questionnaire, codebook), consults the rules database and historical precedents, and drafts a Harmonization Specification — a structured document that proposes exactly how each raw variable should be mapped and transformed.

This AI does not make final decisions. It produces a first draft. A human expert reviews and approves every specification before anything is executed.

3.3.2 Why We Cannot Use Public AI Services

Household survey microdata is sensitive and subject to WBG data classification policies. Sending such data to a public commercial AI service (such as OpenAI's public API) would mean transmitting data to an external server with no institutional data agreement and no guarantee about data residency or retention. This is almost certainly not permissible under WBG policy.

3.3.3 The Enterprise Solution: Azure OpenAI Service

Microsoft offers Azure OpenAI Service — access to the same AI models used in commercial products (GPT-4 and equivalents), but running entirely within Microsoft's Azure infrastructure under a formal enterprise data agreement. Key guarantees include:

- Data does not leave the Azure environment.
- Data is not used to train future models.
- Data residency and retention comply with institutional requirements.
- The service operates under the same security and compliance framework as the rest of our Azure environment.

This is not a workaround. It is the enterprise-grade version of the AI service, built specifically so that governments, international organizations, and regulated industries can use advanced AI without compromising data security.

3.3.4 Important Nuance on Data Sent to the AI

The AI primarily reads survey documentation — questionnaires, codebooks, DDI metadata — not raw respondent-level microdata. This distinction may be relevant to ITS's assessment of what is permissible. We will need ITS's guidance on exactly what categories of data may be sent to Azure OpenAI Service and under what conditions.

3.3.5 Fallback Option

If Azure OpenAI Service is not approved, we would need to explore open-source language models (such as those available through Ollama or similar frameworks) deployed entirely on WBG-controlled infrastructure. This is a significantly more complex setup and may affect system performance. We hope to avoid this path, but we are prepared to discuss it if needed.

> **! ITS QUESTION 3 — PRIORITY**
> Is Azure OpenAI Service approved for use in the WBG Azure tenant? If yes: what data classification levels are permitted, and are there restrictions on what data may be sent to it? If no: what is the approved path for using large language models within the WBG environment?

3.4 Component 4 — Historical Decisions Database

"The system's institutional memory — past decisions that inform future ones"

3.4.1 What It Does

This database stores structured records of every past harmonization decision: what the source variable was, how it was transformed, which GMD variable it was mapped to, what situation was declared, and — critically — whether a human expert overrode the initial recommendation and why. The AI consults this database when drafting specifications for new surveys, using past decisions as informed precedents.

3.4.2 Why It Is Different from the Rules Database

The rules database (Component 2) stores official definitions — what things *must* be. This database stores institutional history — what we *did* before and why. It is advisory, not authoritative. When it conflicts with the rules database, the rules database always wins. But it is enormously valuable because it encodes years of expert judgment in a form the AI can query.

3.4.3 Why This Requires a Special Kind of Database

The rules database can be queried by exact match: "give me the definition of variable X." This database needs to support fuzzy, similarity-based retrieval: "find me past decisions about education variables from surveys in Sub-Saharan Africa where the raw variable had six response categories." That kind of search requires a **vector database** — one that converts the meaning of each record into a mathematical representation and finds the closest matches to a query. This is the technical mechanism behind what is commonly called RAG (Retrieval-Augmented Generation).

3.4.4 What Will Seed This Database

We are not starting from scratch. We have three existing sources of high-quality historical data:

- **Enriched DDI files** from previous harmonization projects, where our `do2screen` tool has already embedded harmonization logic alongside the raw survey metadata.
- **Legacy Stata harmonization scripts** from past GMD waves, which can be parsed and interpreted to extract the decisions they encode.
- **Going forward,** every approved Harmonization Specification automatically becomes a new record in this database — creating a continuous learning loop.

3.4.5 Azure Service

Azure AI Search with vector search capabilities. This service supports both keyword-based and similarity-based retrieval, and is designed for exactly this kind of institutional knowledge retrieval use case.

> **ITS QUESTION 4**
> Is Azure AI Search available and approved in our environment? Can it be configured with vector search capabilities? Alternatively, is there another approved service for similarity-based document retrieval?

3.5 Component 5 — Compute Environment

"The factory floor that runs the pipeline — where our code actually executes"

3.5.1 What It Does

The compute environment is where all pipeline code runs: fetching DDI files, processing documents, calling the AI service, executing approved specifications, running validation checks. It is the machinery that makes all the other components work together.

3.5.2 Why a Virtual Machine Is Not the Right Fit

A traditional virtual machine (VM) is a computer that runs continuously, 24 hours a day, waiting for work. For a process like survey harmonization — which happens in bursts, triggered by discrete events — a permanently-on machine wastes money and resources. We need something that starts when there is work to do and stops when there is not.

3.5.3 Target Architecture: Azure Kubernetes Service (AKS)

AKS is Microsoft's managed platform for running containerized workloads — code packaged with everything it needs to run consistently, deployed and scaled automatically, and shut down when idle. It is the standard enterprise approach for exactly our use case: event-driven, bursty, multi-component pipelines that need to scale without continuous compute costs.

AKS also supports the key requirement that our team can deploy and manage our own code independently, without requiring ITS involvement for every update.

> ⚠ **EXPLICIT REQUIREMENT — AKS IS THE TARGET**
> We understand that a full AKS deployment may not be the fastest path to a proof of concept. We are willing to start with a simpler interim compute environment (such as Azure Container Apps or another lightweight option ITS recommends) for the initial PoC phase. However, we want to be explicit that **AKS is our intended production architecture**. We ask ITS to confirm this path is available, and to ensure that any interim solution is structured as a stepping stone toward AKS — not a permanent replacement for it. We have institutional experience with ITS environments and understand that it is difficult to change infrastructure once it is in place.

3.5.4 What "Team Access" Means Concretely

For the GMD team to operate this system independently, we need the following — regardless of whether we start on AKS or an interim environment:

- The ability to **deploy updated code** to the environment ourselves, without filing an ITS request for each deployment.
- **Read access to job execution logs**, so we can debug problems and verify behavior without asking ITS to look inside the system on our behalf.
- A **sandboxed development environment** where we can test new code against non-production data, completely isolated from any real survey microdata.

ITS QUESTION 5
What is the approved compute environment for running a team-managed, Python-based pipeline that accesses Blob Storage, a SQL database, an AI service, and a vector search index? Our target is AKS. If a simpler environment is recommended for the PoC phase, can ITS confirm that AKS remains the available path forward, and help us understand the migration process? Can our team have self-service deployment and log access?

3.6 Component 6 — Audit Trail and Monitoring

"The flight recorder — a complete record of everything the system did"

3.6.1 What It Does

For every harmonization job the system runs, the audit trail records: what inputs were used and their versions, which Harmonization Specification was approved and by whom, which version of the execution engine ran, what the output was, what the quality assurance checks found, and any human overrides with their justifications.

3.6.2 Why It Is Non-Negotiable

This system produces data that feeds into official World Bank poverty and welfare statistics. At any point in the future, a stakeholder may ask: "How was this variable harmonized in this survey, and why?" The answer must be retrievable, complete, and unambiguous — not approximate. The audit trail is what makes the system accountable and the outputs defensible.

It also protects the team. If a decision is later questioned, the audit trail shows exactly what the AI proposed, what the human changed, and who approved the final version.

3.6.3 Azure Service

This component is the least exotic of the six. Structured run records (what ran, when, what inputs, what outputs) can be stored in the same SQL database as the rules (Component 2). Detailed execution logs go into Azure Monitor or Log Analytics — a standard Azure service that ITS almost certainly has running already. We do not need to build this; we need to ensure our pipeline jobs are connected to whatever logging infrastructure already exists.

ITS QUESTION 6
What is the standard logging and monitoring service in our Azure environment? Can our pipeline jobs be onboarded to it? What is the standard data retention policy for logs? Our team will need read access to query logs directly, without routing requests through ITS.

---

4 Consolidated Questions for ITS

The following table collects all questions from the sections above. We ask that ITS provide a written response to each before our follow-up meeting, so that we can make concrete infrastructure decisions and move the proof of concept forward.

Table 2: Consolidated ITS Questions

#	Component	Question
Q1	Storage	Is there an approved Azure Blob Storage environment for restricted household survey microdata? What access controls and data classification requirements apply?
Q2	Storage / API	Can our compute environment make outbound API calls to `microdata.worldbank.org` to retrieve DDI metadata files?
Q3	Relational DB	What is the standard approved relational database service in our Azure environment (Azure SQL, PostgreSQL, other)? Is there an existing instance available?
Q4	**AI Service**	**PRIORITY — Is Azure OpenAI Service approved in the WBG Azure tenant? If yes: what data classification levels are permitted? If no: what is the approved path for using large language models in our environment?**
Q5	Vector Search	Is Azure AI Search available and approved in our environment with vector search capabilities enabled? If not, is there an alternative approved service for similarity-based document retrieval?
Q6	Compute — Target	Is Azure Kubernetes Service (AKS) available for a team-managed workload? Can our team have self-service deployment and log access?
Q7	Compute — PoC	If AKS is not immediately available, what is the simplest approved compute environment for running scheduled Python-based jobs? Can ITS confirm AKS remains the path for production?
Q8	Monitoring	What is the standard logging service in our environment? Can our jobs be onboarded? What is the retention policy? Can our team query logs directly without routing requests through ITS?

---

5 What We Are Not Asking ITS to Build

To be explicit: we are not asking ITS to understand, design, or build the AI pipeline itself. ITS does not need to understand what a Harmonization Specification is, how the AI reasons about education variables, or how the historical precedents database is populated.

We are asking ITS to provision, secure, and connect the six infrastructure components described above — and to configure them so that our team has the access needed to build and operate the pipeline independently.

The GMD team will own and operate everything that runs inside that infrastructure. ITS owns the infrastructure itself.

---

6 Proof of Concept Scope

The initial proof of concept (PoC) covers **non-welfare demographic variables only**: gender, age, education, urban/rural classification, household relationships, and labor force status. Welfare aggregate variables (consumption, income) are explicitly out of scope.

Volume: approximately 100–200 new surveys per year. The PoC will begin with a representative subset of these.

The PoC is intended to prove that the methodology works — not to build production-grade infrastructure from day one. We are therefore open to starting with simplified versions of some components (particularly compute) if that allows us to move faster. However, all infrastructure decisions should be consistent with the target architecture described in this document.

GMD Harmonization Team · Development Data Group · World Bank Group · May 2026 · DRAFT v1.0